```cpp
#pragma once
#include <cmath>
#include "matrix.hpp"

class LayerNorm {
private:
    size_t dim_;
    double eps_;

public:
    LayerNorm(size_t dim, double eps = 1e-5) : dim_(dim), eps_(eps) {}

    // Normaliza cada columna (vector de token) de la matriz X
    Matriz forward(const Matriz& X) {
        Matriz Y(X.filas(), X.columnas());

        for (size_t col = 0; col < X.columnas(); ++col) {
            // 1. Calcular Media
            double media = 0.0;
            for (size_t fila = 0; fila < X.filas(); ++fila) {
                media += X(fila, col);
            }
            media /= X.filas();

            // 2. Calcular Varianza
            double varianza = 0.0;
            for (size_t fila = 0; fila < X.filas(); ++fila) {
                double diff = X(fila, col) - media;
                varianza += diff * diff;
            }
            varianza /= X.filas();

            // 3. Normalizar
            for (size_t fila = 0; fila < X.filas(); ++fila) {
                Y(fila, col) = (X(fila, col) - media) / std::sqrt(varianza + eps_);
            }
        }
        return Y;
    }
};
```